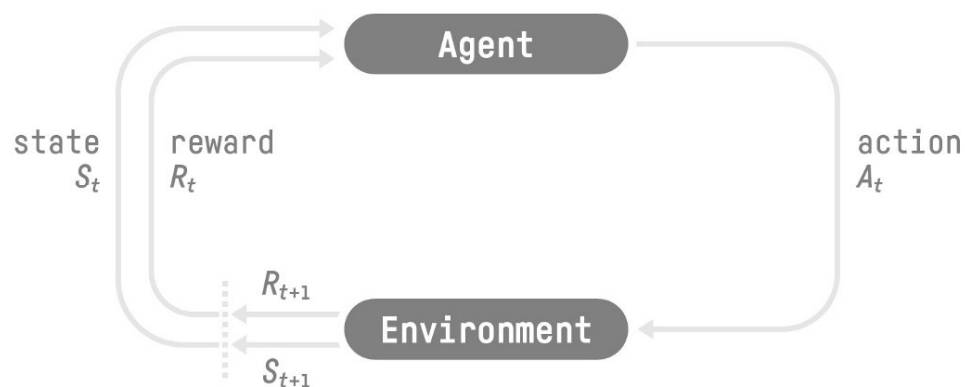


Reinforcement Learning

Naf Guo
2026

What is Reinforcement Learning



我们的目标是构建一个代理agent，能够与环境交互并采取智能决策。

其中，环境按不同的时间 t 处于不同的状态 S_t 。我们的代理agent根据所处的环境状态采取行动 A_t 。采取行动会带来两个效果，首先环境的状态会因为和agent的交互发生改变为 S_{t+1} ；其次agent会获得奖励reward R_{t+1} 。我们的目标是最大化奖励的累积，称为总回报return。

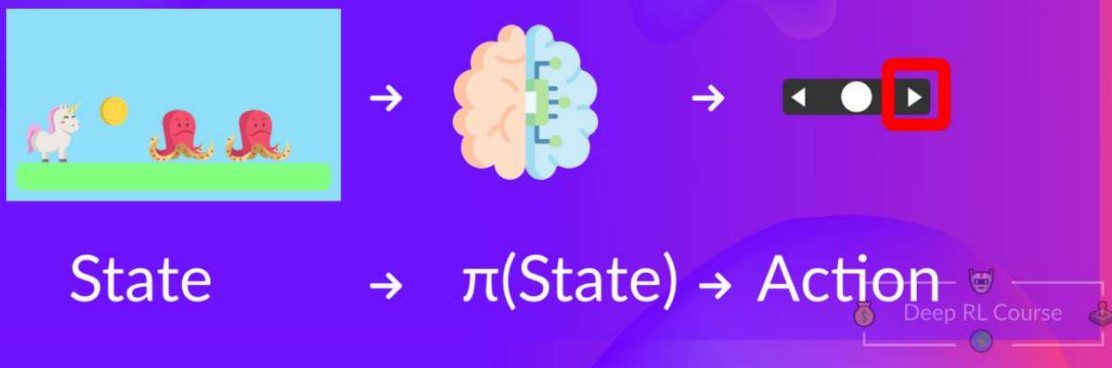
return 是从当前时刻到终止时刻的奖励总和，因此只有过程结束后才能得到一个确定的数值。

策略 Policy

The Policy π : the agent's brain

Policy π : is the brain of our Agent, it's the function that tell us what action to take given the state we are.

→ So it defines the agent behavior at a given time.



策略控制着代理的行为。

也就是我们的agent在环境处于状态S时，要采取什么行为。

强化学习的目标是要找到一种最优策略，使我们获得的期望回报最大。

也就是说，策略是一种关系，给定环境的状态，返回代理接下来采取的行动，或者行动分布

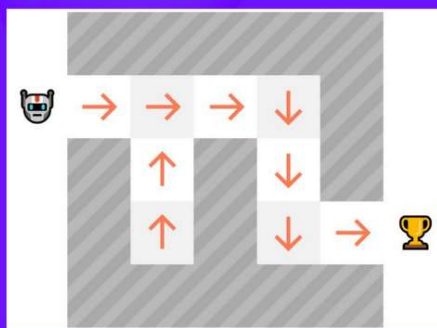
$$a_{t+1} = \pi(s_t) \quad \text{or} \quad a_{t+1} \sim \pi(\cdot | s_t)$$

我们希望找到最优策略 π^* ，使得能获得的累积奖励最大

如何找到最优策略

Two approaches to find optimal policy π^* :

Policy-Based methods: train the agent to learn which action to take, given a state.



Value-Based methods: train the agent to learn which state is more valuable and take the action that leads to it.



考察一个游戏，从面具处出发，每步能移动一个格子，每走一步会有-1的reward，而到达奖杯处能获得+1的reward。

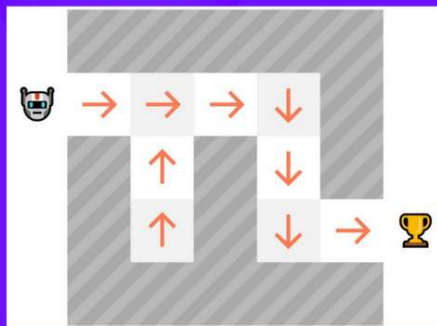
游戏的目标就是达到奖杯处。

怎么找最优的策略？

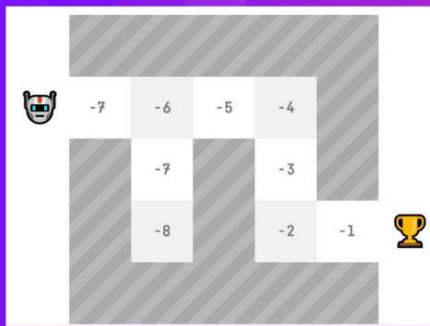
如何找到最优策略

Two approaches to find optimal policy π^* :

Policy-Based methods: train the agent to learn which **action to take**, given a state.



Value-Based methods: train the agent to learn which state is **more valuable** and take the action that leads to it.



一般分两种

- 基于策略的方法，agent学习的是一套策略，在不同状态应该采取什么行动，如图左，agent到达每个状态以后，箭头代表它要采取的行动，基于策略的方法就是学习在每个位置应该画什么箭头；
- 基于价值的方法，这种方法会对每一个环境状态分配一个分数，分数代表那个状态更有价值，最终能带来更大的回报，而agent的最优行为就是每次都向累积回报(状态价值分数+单步回报)最高的状态移动。

图左，学到一套策略，在每个状态应该怎么走才能到达奖杯

如图右，每个格子上的数字代表这个状态的价值(这里的状态就是指代理到达某一个格子)，数字越大代表这个状态的价值越大，而代理的最优策略就是顺着价值增加的方向一路 -7 -6 -5 -4 -3 -2 -1 到达奖杯。

基于价值的方法

Value-Based Methods

The value of a state is the expected discounted return the agent can get if it starts in that state, and then act according to our policy.

$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s]$$

Value function Expected discounted return Starting at state s

在上一页我们说，在基于价值的方法中，我们要为每一个状态分配一个价值，那这个价值应该怎么分配呢？

实际操作中，我们给它的分配是，从该状态出发，采取策略 π 走完剩余过程，能获得的回报的期望。

（这里用期望是因为，策略可能带有随机性，即处于某个状态，采取行动1234的概率分别是abcd云云）

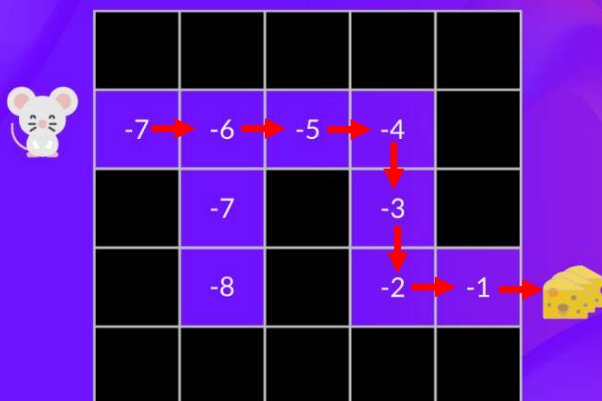
一定要注意，价值分配是和采取的策略相关的

- 价值函数 $V_{\pi}(s)$ 是在策略 π 下定义的，它衡量的是“按这个策略走，这个状态好不好”。
- 当我们有了价值函数后，就可以用贪婪策略，即每次都采取价值最大的行动来生成一个完整的策略 π

贪婪策略

The State Value Function

State Value Function: calculate the value of a state.



假设我们有了一套价值函数，就是图上标示的数字。

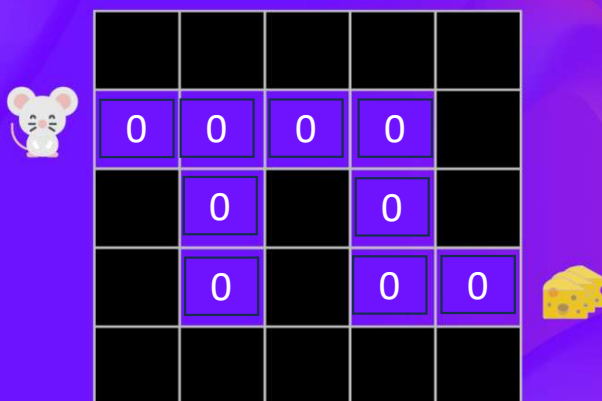
- 到达奶酪处则游戏结束
- 而每走一步都会有-1的reward(这样强迫算法找到最短路线)
- 那走到某个格子上的回报Return就是该格子的价值分+走的那一步的reward
- 如从-6到-5的格子，return就是 $-5 + (-1)$

贪婪策略就是，每次都只采取能使return最高的行动

状态价值函数

The State Value Function

State Value Function: calculate the value of a state.



$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s]$$

Value function Expected discounted return Starting at state s

一定要注意，**return** 是从当前时刻到终止时刻的奖励总和，因此只有过程结束后才能得到一个确定的数值。

我们回到刚才的例子，看看这些值函数怎么算出来。

先把所有状态的价值初始化为0

为了使得老鼠采取的行动数目尽可能小，每走一步，都在reward里加入一个-1的惩罚；作为无良的人类，我放的奶酪其实是假的，因此到达奶酪的奖励为0，但一旦到达，整个过程就结束了。

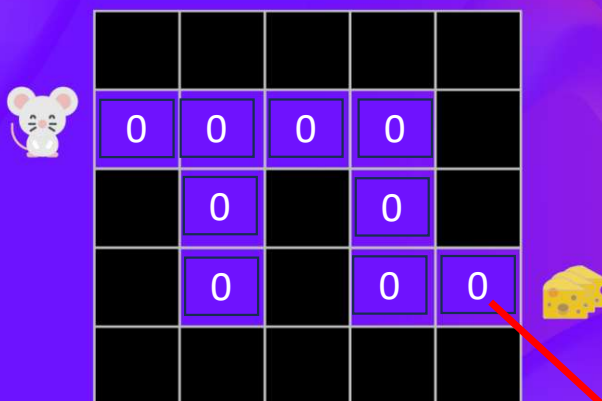
设置 $\gamma = 1$ ，也就是不折扣。

小老鼠采取贪心策略，也就是只会选择价值最大的动作。

状态价值函数

The State Value Function

State Value Function: calculate the value of a state.



$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s]$$

Value
function

Expected discounted return

Starting
at state s

一定要注意，**return** 是从当前时刻到终止时刻的奖励总和，因此只有过程结束后才能得到一个确定的数值。

我们可以手动检查一下所有可能性，给每个位置安排上价值分数。

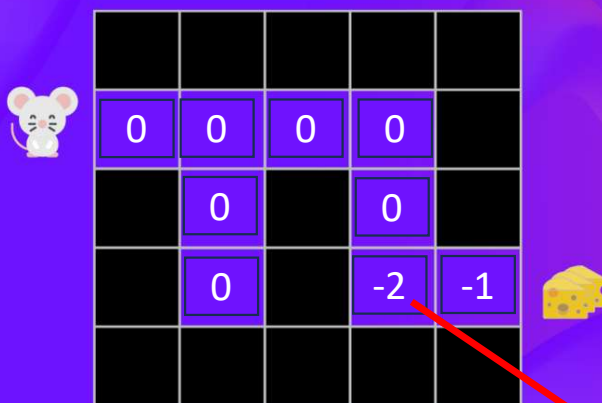
小老鼠从这个格子出发，如果向右走，则过程结束，return是-1+0=-1;

如果向左走，则每走一步有-1的罚分，但走了以后这个过程没结束，想结束必须走回头路，那又是-1的罚分，直到走到奶酪才能结束，则向左走的return是肯定比向右走的return小的。我们采取贪婪策略，所以向右走，return为-1，那这个格子的价值是-1

状态价值函数

The State Value Function

State Value Function: calculate the value of a state.



$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s]$$

Value function Expected discounted return Starting at state s

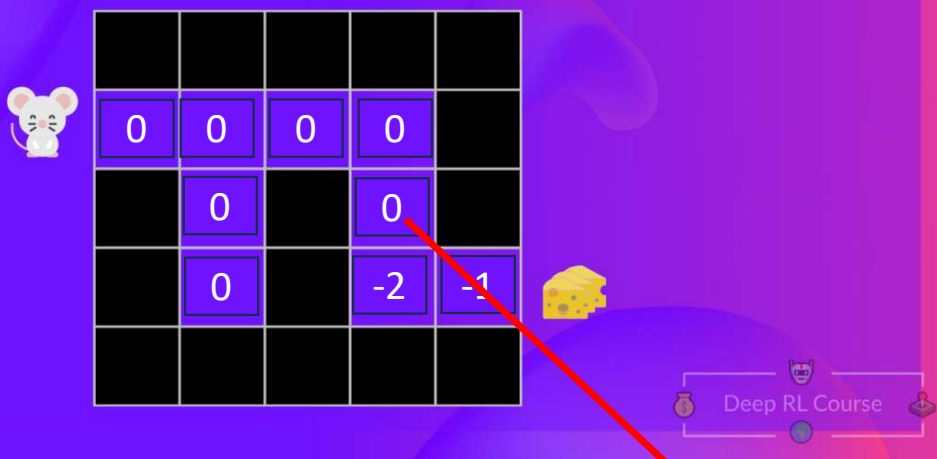
同样地，去看这个格子，它能获取最大return的方法是往右走2步到达奶酪，return是 $-1 + -1 = -2$ ，所以这个格子的价值是-2

一定要注意，**return** 是从当前时刻到终止时刻的奖励总和，因此只有过程结束后才能得到一个确定的数值。

状态价值函数

The State Value Function

State Value Function: calculate the value of a state.



$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s]$$

Value function Expected discounted return Starting at state s

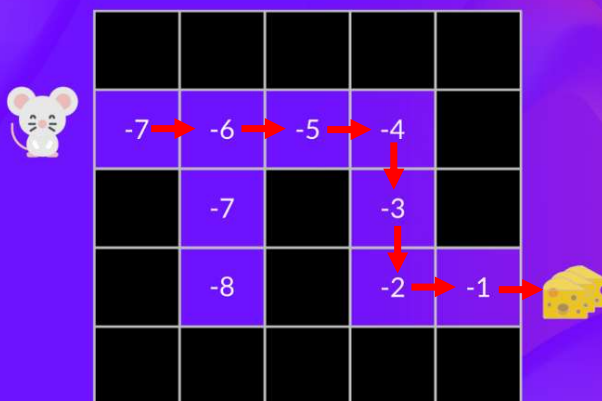
这个格子，它能获取最大return的方法是向下走1步，往右走2步到达奶酪，return是-3，所以这个格子的价值是-3

一定要注意，**return** 是从当前时刻到终止时刻的奖励总和，因此只有过程结束后才能得到一个确定的数值。

状态价值函数

The State Value Function

State Value Function: calculate the value of a state.



$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s]$$

Value function Expected discounted return Starting at state s

总之在贪心策略下，我们把每个格子的价值都算出来了

之后最优策略就是顺着箭头走

一定要注意，**return** 是从当前时刻到终止时刻的奖励总和，因此只有过程结束后才能得到一个确定的数值。

状态价值函数 The state-value function

Two types of Value-Based Methods

State Value Function:

$$V_{\pi}(s) = \mathbf{E}_{\pi}[G_t | S_t = s]$$

Value of state s

Expected return

If the agent starts at state s

And uses the policy to choose its actions for all time steps

For each state,
the state-value function outputs
the expected return
if the agent starts in that state
and then follows the policy forever after.

Deep RL Course

在上面的方法中，我们的价值函数只和状态 s 相关，称为状态价值函数，这种价值函数用 V 表示

动作价值函数 The action-value function

Two types of Value-Based Methods

Action Value Function:

$$Q_{\pi}(s, a) = \mathbf{E}_{\pi}[G_t | S_t = s, A_t = a]$$

Value of state-action pair s, a

Expected return

If the agent starts at state s

and chooses action a

And then uses the policy to choose its actions for all time steps

For each state and action, the action-value function outputs the expected return if the agent starts in that state and takes the action and then follows the policy forever after.

Deep RL Course

还有一种价值函数称为Q函数，它是在状态 s 下，且采取动作 a ，将来能得到的回报。就是多规定了一个动作。

基于价值的方法

The link between Value and Policy:

$$\pi^*(s) = \arg \max_a Q^*(s, a)$$

Finding an optimal value function leads to having an optimal policy.

在强化学习中，无论用哪种方法，最终都会得到一个策略。基于策略的方法是直接对策略本身进行训练，从而找到最优策略 π^*

而基于价值的方法则不直接训练策略，而是先学习出最优价值，再通过“在每个状态下选择能让动作价值最大的行为”这一规则（如贪心策略）推导出最优策略，在实际应用中，为了平衡探索与利用，我们通常会使用 ϵ -贪心策略来替代纯贪心策略。